

ChangeCheck · 改版验收官

AI 功能的回归验收层 —— 改了提示词、换了模型之后，用证据回答：哪些变好了？哪些被改坏了？这次改版值不值得上线？

参赛信息：2026 上海开源软件应用创新大赛 · 开源 AI 工具赛道（自主选题） | 项目形式：独立开源 Web 应用（MIT 协议） | 仓库：github.com/nghqqa/changecheck

0. 三十秒看懂

AI 功能的迭代方式是改提示词、换模型、调流程——这也是所有软件里回归风险最高的一类改动：输出不确定、无法断言，个人开发者只能改完自己试两三条、看着流畅就上线。一次「好心」的改版（比如要求输出更完整），模型就开始给没写的字段编造 18:00、给"另行通知"编造截止时间——而平均观感还在变好，新增的坏被完全掩盖。

ChangeCheck 把它变成一道有证据的验收工序：**业务要求** → **可确认的检查项** → **回归测试集** → **两版本对比运行** → **新增失败优先的验收报告**。传统代码有 CI 挡回归，AI 功能此前没有——ChangeCheck 补上这一层。

36/36

植入退步全部抓到（M0 验证实验，真实 API）

0 误报

该批次 36 条稳定样例误报为零

8 条

盲测中真实抓出的新增失败（未预埋结果）

35 项

内核/CLI 单元测试，CI 验收看门全绿

1. 背景与痛点

LLM 应用已进入“频繁改版”阶段，但没有配套的回归验收手段。接入大模型的功能（问答、提取、分类、生成）在上线后会被持续修改：调提示词、换更便宜的模型、改流程。每一次修改都是对“AI 行为”的全量改动——没有任何测试覆盖改后的行为，因为输出是不确定的。

现状做法与代价：

- **肉眼抽样：**改完自己试 2-3 条输入，看着不错就发布。样本外的行为变化未知。
- **平均分误导：**即使做了评测，大家看的也是平均分/通过率——平均分上涨完全可能与“某类输入出现新增失败”同时发生。
- **错误不积累：**线上发现一条坏输出，反馈给开发者看一眼就过去了，不会沉淀为回归用例；同一个错误在下次改版后复发。

真实复现（本项目实验数据，非虚构）：我们用 DeepSeek (deepseek-flash) 做了两组实验——

① **植入缺陷演示：**把稳定提示词改成"输出更完整可用、未写字段给合理默认值"（一个典型的好心改版）。结果候选版 36 条样例全部出现关键违规：给"具体时间另行通知"补 18:00、给未定地点填"待定"、把"下周五"擅换成具体日期，且费用 +215%、耗时 +205%。

② **盲测：**改用一个看似无害的"精简提示词省 token"版本（**未预埋任何缺陷，结果事先未知**）。工具真实抓出 **8 条新增失败（8 条关键违规）**：因删掉"date/deadline 区分"规则导致 5 条样例的日期错误落入 deadline 字段、给"延期待定"补字面"待定"——而且更贵 +73%、更慢 +81%（指令模糊使思考型模型消耗更多推理 token）。"以为省了钱，实际又贵又坏"，只有逐条证据能揭示。

2. 市场定位与差异化

我们对 11 个主流产品做了逐项核查（2026-10，官网/GitHub/定价页，见仓库 docs/market-research.md）。格局要点：promptfoo 已被 OpenAI 收购、Langfuse 已被 ClickHouse 收购，头部开源"名花有主"；平台型产品（LangSmith/Braintrust 等，\$39-249/月）集体服务团队与企业；「**个人开发者 × 中文场景 × 本地一条命令 × 回归优先**」是确认的市场空白。

能力维度	行业现状	ChangeCheck
两版本对比运行	已被满足 (promptfoo/Langfuse 等)	✅ 支持 (同模型不同提示词 / 不同模型)
回归优先报告	主流产品中未见一等公民 (2026-10 检索)：均以平均分/通过率仪表盘为中心	✅ 核心差异： 新增失败 → 已修复 → 仍失败 → 通过率；关键违规标红；三态结论 (采用/复核/拒绝)
自然语言要求 → 检查项	仅存在于安全红队域 (promptfoo redteam)；业务质量向无人做	✅ 要求引导引擎： 把"没写的留空、不能编"变成可勾选确认的检查项；自动检出"补全类"要求与忠实性冲突
轻量本地 × 错误积累	互斥：错误回流强的都要 SDK/后端/SaaS；轻量的无生产采集	✅ 零运维积累： 线上坏输出一键入库为回归用例 (origin=real-error)，每次改版自动复检
中文与国产模型	检索范围内未见同类 (除模型榜单与云厂商服务)	✅ 中文优先断言 (数字/日期保护、分词容忍、命名等价) + 全中文界面与报告；OpenAI 兼容协议直连 DeepSeek/GLM/Kimi
中立性	promptfoo→OpenAI、Langfuse→ClickHouse	✅ 独立中立开源 (MIT)，无厂商背景

3. 应用场景

3.1 首发场景：中文通知结构化信息提取（已完整实现并验证）

"通知转待办/日程"类功能：从中文通知中提取活动名、日期、时间、地点、截止时间。字段与值明确可机器核验、错误一眼可见。内置 36 条四类种子测试集（正常/缺失/歧义/对抗），开箱即可复现本页所有实验数

据。

3.2 异构扩展场景（数据模型已预留）

- **客服/工单分类**：改提示词后分类错误率是否在某类工单上恶化（枚举字段 + 参考答案模式相同）
- **文档问答/摘要**：关键信息是否漏答、是否开始编造（要求引导生成"必须包含/禁止编造"检查项）
- **任何"输入文本 → 结构化输出"的 AI 功能**：任务 schema 与检查规则均为数据驱动，新增场景不改内核

3.3 目标用户

已在产品中接入 AI 能力的个人开发者与小团队：不想搭建评测平台、不会写评测 YAML，只想在改版上线前知道"有没有改坏"。对比之下，promptfoo/DeepEval 面向会写配置的工程师，Langfuse/Braintrust 面向有平台的团队。

4. 技术架构

单体 Next.js 应用 + SQLite（Node 内建 node:sqlite，零原生依赖）+ OpenAI 兼容 API + Docker Compose 一键部署。关键设计是**内核与 UI 解耦**：内核三层不依赖 Next/React，为 v2 的 CLI/CI 守门与 MCP server（AI 编程代理自跑验收）直接复用而设计。

层	组件	职责与要点
内核 (自研)	执行器 executor	并发批量调用、重试退避、单次调用硬超时、用量/费用/耗时逐条记录、多次重复 + 多数决（吸收 temperature=0 下的真实波动）
	检查器 checker	JSON/schema/字段级规则：缺失必须留空（凭空补全恒为关键违规）、必须提取、逐字一致（含数字保护，防"10月1日<10月17日"假近似）、名称等价放宽（首个主体字段）；语义反转防护（不/取消/以外/以后等不对称即判失败）。规则全部经 35 项单元测试固定
	回归报告 report	新增失败优先、关键违规标红、分类统计、费用/耗时对比、三态结论（含 punchline 叙事）、Markdown 导出；逐条证据存 run_items，报告可追溯
要求 引导	requirements	自然语言要求 → 检查项草稿（确定性解析，零 API 成本）→ 人工勾选确认 → 固化为任务验收标准，进每份报告；自动检出"补全 vs 留空"类要求冲突
编排	runs	串行运行队列（防并发轰 API）、进度逐条持久化、报告为运行时快照（样例事后编辑不影响历史）
数据	SQLite 五表	tasks/samples/versions/runs/run_items；检查项结构化存储、样例带 origin（seed/manual/real-error，为错误回流预留）、run 与 task 分离（为趋势回溯预留）
UI	Next.js App Router	四步向导（验收标准→测试集→版本→报告）；无 key 可用 mock 模式跑通全流程

4.1 工程化

- **测试**：35 项单元测试（内核判定 / 要求解析 / CLI 接入与退出码语义），`npm test`；GitHub Actions 双流水线：测试+构建、以及「验收看门」（用本工具守本仓库，报告归档为 artifact）
- **判定口径 v0.6**：字段级关键违规一票否决（防多数决掩盖与新旧错误交换）、调用大面积失败自动判「结果不可信」、语义反转防护（不/取消/以外/以后等）——判定规则随版本演进，本文实验数据注明所属口径
- **可靠性**：单次调用 180s 硬超时与重试；思考型模型 `max_tokens` 截断检测（实测沉淀）；串行队列 + 进度可视化
- **安全**：API key 仅经环境变量注入，不落库不进仓库；SQL 全部参数化；数据本地 SQLite
- **可复现**：首次打开自动播种三个演示任务、种子测试集与**四份可打开的历史演示报告**（含真实 API 运行），评审 `docker compose up` 即可复现本页全部数据，无需先跑一次

5. 创新点小结

1. **回归优先的验收报告**（主流产品未见同类实现，2026-10 检索）：以“这次改版新增了哪些失败”为第一视图，证据链到字段级——违反哪条要求、期望/实际值、原文依据；平均分叙事被明确替换为反差叙事（“费用 -68%，但有 8 条新增失败”）。
2. **要求引导引擎**（把 promptfoo 红队已验证的“自然语言→用例”交互迁移到业务质量域）：不懂评测的人用一句话描述要求，得到可确认的检查项；冲突检测在引导阶段就拦截“为完整而补全”这类高发事故源。
3. **零运维的测试资产积累**：线上坏输出一键入库为回归用例，测试集随产品长大——这是用户迁不走的资产，也是本项目主张的护城河（执行引擎是 commodity，验收资产不是）。
4. **中文优先与国产模型一等公民**：中文断言细节（数字/日期保护、命名等价、全半角归一）、全中文产品与报告、OpenAI 兼容直连 DeepSeek/GLM/Kimi——2026-10 检索范围内未见同类实现。

6. 开源治理

- MIT 协议，无厂商背景的中立开源；依赖极简（运行时仅 `next/react/react-dom`，数据库用 Node 内建模块）
- 文档体系：README（快速开始/环境变量/结构）、CONTRIBUTING（约定：内核禁 UI 依赖、规则改动必须配测试）、Issue 模板、ADR 决策记录（docs/project-brief.md, D1-D7）、架构文档、市场调研
- CI：GitHub Actions 测试 + 构建；提交历史清晰，每个功能对应一个可读 commit
- 隐私与安全边界明确：key 只走环境变量；所有数据本地存储

7. 发展规划

阶段	内容
v1.5 (赛后数周)	AI 生成测试草稿 (人工确认制) ; AI-as-judge (中文校准) 抽样审计; 分类/问答场景模板; 重复运行波动分析
v2 (数月)	MCP server (AI 编程代理改完提示词自跑验收) ; CLI + CI 守门已于 v1 提供 (<code>changecheck run</code> , 退出码语义: 仅新增失败非零, 本仓库 CI 已实装) ; 测试集进 Git 的多人协作; GLM/Qwen/Kimi provider 预设
v3 (愿景)	测试资产生态 (模板包、共享验收标准) ; 按用户验收标准做模型选型推荐 ("全过验收前提下最便宜/最快")

本介绍文档中的全部实验数据来自项目仓库内可复现的运行记录 (Mileston 0 验证实验 + 盲测) , 评审可通过 Docker 一键部署验证。